Architecture and scale

3B. Architecture & Scale

Questions: 4 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Atlassian, Cred

These questions test whether you've thought about CSS beyond single components. How do you organize styles when 10 engineers are working on the same codebase? How do you prevent one team's CSS from breaking another team's UI? How do you support dark mode, multiple brands, and responsive layouts without duplicating code?

Q 3.6 Compare CSS architecture approaches — BEM, CSS Modules, CSS-in-JS, Tailwind. What are the trade-offs at scale?

Difficulty: Mid **Asked at:** Razorpay, Flipkart, Swiggy, Atlassian, Cred

Every CSS architecture approach solves the same core problem: how to write CSS that doesn't break when the codebase grows. They solve it differently, and each has trade-offs.

BEM (Block Element Modifier):

```css
/* Block */
.card { }
```

```
/* Element (part of the block) */
.card__body { }
.card__footer { }

/* Modifier (variant of block or element) */
.card--featured { }
.card__title--large { }
```

HTML Copy

```
<div class="card card--featured">
  <h2 class="card__title card__title--large">Featured Product</h2>
  <p class="card__body">Description here</p>
</div>
```

Pros: No build tools needed. Flat specificity (everything is a single class). Naming convention makes relationships clear. Works everywhere.

Cons: Verbose class names. No scoping enforcement — naming discipline is the only thing preventing collisions. Gets tedious in large codebases. HTML gets cluttered with long class names.

When to use: Projects without a build tool, teams that want simplicity, WordPress/CMS themes, email templates.

CSS Modules:

CSS Copy

```
/* Card.module.css */
.card { padding: 16px; border-radius: 8px; }
.title { font-size: 1.25rem; font-weight: 600; }
.featured { border: 2px solid gold; }
```

JSX Copy

```
import styles from './Card.module.css';

function Card({ featured, title }) {
```

```
    </div>
  );
}
```

The build tool (Vite, webpack) transforms class names to unique hashes: `.card` becomes `.Card_card_x7f2k`. Collisions are impossible.

Pros: True scoping — styles can't leak between components. Write plain CSS (no new syntax). Full browser DevTools support. Co-located with components. Zero runtime cost.

Cons: Dynamic styling requires conditional class joining (can get messy). No access to JS variables in CSS (use CSS custom properties instead). Class name composition across files is awkward.

When to use: React/Vue/Svelte projects where you want scoped CSS without a runtime library. The most balanced choice for most teams.

CSS-in-JS (styled-components, Emotion):

JSX Copy

```jsx
const Card = styled.div`
  padding: 16px;
  border-radius: 8px;
  border: ${props => props.$featured ? '2px solid gold' : '1px solid #eee'
`;

const Title = styled.h2`
  font-size: ${props => props.$large ? '1.5rem' : '1.25rem'};
  color: ${props => props.theme.colors.text};
`;

function ProductCard({ featured, title }) {
  return (
    <Card $featured={featured}>
      <Title>{title}</Title>
    </Card>
```

Pros: Full access to JavaScript (props, theme, state) inside styles. Automatic scoping. Co-located with components. Dynamic styling is natural.

Cons: Runtime cost — styles are computed and injected at render time. Larger bundle size (library code). Harder to debug in DevTools (generated class names). Performance issues with frequent re-renders. Server-side rendering requires extra setup.

When to use: Applications with highly dynamic styling that depends on runtime state, design systems that need theme-aware components. Becoming less popular in 2026 due to performance concerns and the rise of zero-runtime alternatives.

Tailwind CSS:

HTML Copy

```html
<div class="p-4 rounded-lg border border-gray-200 hover:shadow-lg transiti
  <h2 class="text-xl font-semibold text-gray-900">Product Title</h2>
  <p class="text-gray-600 mt-2 line-clamp-3">Description here ... </p>
  <button class="mt-4 px-4 py-2 bg-blue-600 text-white rounded-md hover:bg
    Buy Now
  </button>
</div>
```

Pros: No naming decisions. No context switching between HTML and CSS files. Tiny production CSS (only includes classes you use). Consistent design system through config. Fast prototyping.

Cons: HTML gets verbose and harder to read. Design changes require touching every instance (no single class to update). Learning the utility class names takes time. Responsive and state variants add complexity (`md:flex lg:grid hover:bg-blue-700 dark:text-white`). Custom designs beyond the default scale need configuration.

When to use: Teams that want speed and consistency. Works exceptionally well with component-based frameworks (React, Vue) where HTML repetition is managed by components.

What interviewers want to hear:

Not "I prefer X." They want you to explain the trade-offs for a given situation. "For a small team building a new product quickly, I'd recommend Tailwind for development speed. For a large design system used by multiple teams, I'd lean toward CSS Modules for scoping guarantees with zero runtime cost. For a heavily themed application with runtime color switching, CSS-in-JS might still make sense, though I'd consider using CSS custom properties to minimize the runtime cost."

Follow-up: "What about zero-runtime CSS-in-JS like Vanilla Extract or Panda CSS?"

These compile CSS-in-JS syntax to static CSS at build time — you get the developer experience of writing styles in TypeScript with zero runtime cost. They're the emerging middle ground and worth mentioning in interviews as an awareness signal.

Q 3.7 What are design tokens? How do you implement theming with CSS custom properties?

Difficulty: Mid **Asked at:** Google, Razorpay, Flipkart, Cred

Design tokens are the atomic values that define a design system — colors, spacing, typography, shadows, border radii, breakpoints. They're the single source of truth that ensures a button in the header uses the exact same blue as a button in the footer, and that changing that blue updates both.

Implementing tokens with CSS custom properties:

```css
:root {
  /* Colors */
  --color-primary: #3b82f6;
  --color-primary-hover: #2563eb;
  --color-secondary: #64748b;
  --color-success: #22c55e;
  --color-error: #ef4444;
  --color-text: #1e293b;
  --color-text-muted: #64748b;
```

```
  /* Spacing */
  --space-xs: 4px;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --space-xl: 32px;
  --space-2xl: 48px;

  /* Typography */
  --font-sans: 'Inter', system-ui, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
  --text-sm: 0.875rem;
  --text-base: 1rem;
  --text-lg: 1.125rem;
  --text-xl: 1.25rem;
  --text-2xl: 1.5rem;

  /* Shadows */
  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.05);
  --shadow-md: 0 4px 6px rgba(0, 0, 0, 0.07);
  --shadow-lg: 0 10px 15px rgba(0, 0, 0, 0.1);

  /* Border radius */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-full: 9999px;
}
```

Components consume tokens, never hardcoded values:

css Copy

```
.button {
  font-family: var(--font-sans);
  font-size: var(--text-sm);
  padding: var(--space-sm) var(--space-md);
  border-radius: var(--radius-md);
  background: var(--color-primary);
```

```
  box-shadow: var(--shadow-sm);
}

.button:hover {
  background: var(--color-primary-hover);
  box-shadow: var(--shadow-md);
}
```

Dark mode with token overrides:

The power of custom properties is that they cascade. Override them on a parent element and all children update automatically:

css Copy

```
/* Light mode (default) */
:root {
  --color-text: #1e293b;
  --color-bg: #ffffff;
  --color-bg-secondary: #f8fafc;
  --color-border: #e2e8f0;
  --shadow-md: 0 4px 6px rgba(0, 0, 0, 0.07);
}

/* Dark mode */
[data-theme="dark"] {
  --color-text: #e2e8f0;
  --color-bg: #0f172a;
  --color-bg-secondary: #1e293b;
  --color-border: #334155;
  --shadow-md: 0 4px 6px rgba(0, 0, 0, 0.3);
}

/* System preference */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --color-text: #e2e8f0;
    --color-bg: #0f172a;
    /* ... same overrides ... */
```

JAVASCRIPT Copy

```javascript
// Toggle theme
function toggleTheme() {
  const current = document.documentElement.getAttribute('data-theme');
  const next = current === 'dark' ? 'light' : 'dark';
  document.documentElement.setAttribute('data-theme', next);
  localStorage.setItem('theme', next);
}
```

Every component that uses `var(--color-bg)` automatically switches between light and dark without any component-level changes. You change the tokens at the root, and the entire UI updates.

Multi-brand theming:

CSS Copy

```css
[data-brand="brand-a"] {
  --color-primary: #3b82f6;    /* Blue */
  --font-sans: 'Inter', sans-serif;
  --radius-md: 8px;
}

[data-brand="brand-b"] {
  --color-primary: #8b5cf6;    /* Purple */
  --font-sans: 'Poppins', sans-serif;
  --radius-md: 16px;
}
```

Same components, different visual identity — controlled entirely through token overrides.

Follow-up: "How do design tokens work across platforms (web, iOS, Android)?"

Tools like Style Dictionary or Figma's token plugin export design tokens to multiple formats — CSS custom properties for web, Swift constants for iOS, Kotlin constants for

Android. The design team maintains one set of tokens, and each platform consumes them in its native format. This ensures the same blue, same spacing, same typography across every platform.

Q 3.8 How do you approach responsive design in 2026?

Difficulty: Mid **Asked at:** Flipkart, Swiggy, Amazon, Google

Responsive design in 2026 is fundamentally different from the breakpoint-based approach that dominated 2015-2020. The shift is from responsive pages to responsive components, from fixed breakpoints to fluid scaling, and from media queries to container queries.

The modern responsive approach has four layers:

Layer 1: Fluid foundations — no breakpoints needed

css Copy

```css
.container {
  /* Fluid width with a max */
  width: min(90%, 1200px);
  margin-inline: auto;

  /* Fluid padding */
  padding: clamp(16px, 4vw, 64px);
}

h1 {
  /* Fluid typography — scales from 24px to 48px */
  font-size: clamp(1.5rem, 1rem + 3vw, 3rem);
}

.grid {
  /* Fluid grid — items wrap naturally */
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(280px, 100%), 1fr));
  gap: clamp(16px, 3vw, 32px);
}
```

Layer 2: Container queries — components adapt to their context

```css
.card-wrapper {
  container-type: inline-size;
}

.card {
  display: flex;
  flex-direction: column;
}

@container (min-width: 500px) {
  .card {
    flex-direction: row;
    align-items: center;
  }

  .card img {
    width: 200px;
    flex-shrink: 0;
  }
}
```

The card switches from vertical to horizontal layout based on its container's width, not the viewport. This means the same card component works in a narrow sidebar and a wide main content area without any external coordination.

Layer 3: Media queries — for truly viewport-dependent changes

```css
/* Navigation pattern changes */
@media (max-width: 768px) {
  .desktop-nav { display: none; }
  .mobile-nav { display: block; }
}
```

```
  }
}
```

Media queries still have a role — but for page-level layout changes and navigation patterns, not for component sizing.

Layer 4: User preferences

```css
@media (prefers-color-scheme: dark) { /* Dark mode */ }
@media (prefers-reduced-motion: reduce) { /* Reduce animations */ }
@media (prefers-contrast: more) { /* High contrast */ }
@media (hover: none) { /* Touch devices — larger tap targets */ }
```

Mobile-first vs desktop-first:

Mobile-first means your base CSS is the mobile layout, and you use `min-width` media queries to add complexity for larger screens:

```css
/* Base: mobile layout */
.grid { grid-template-columns: 1fr; }

/* Add complexity for larger screens */
@media (min-width: 768px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .grid { grid-template-columns: repeat(3, 1fr); }
}
```

Mobile-first is the standard approach because it's easier to progressively add layout complexity than to strip it away. It also tends to produce smaller CSS for mobile devices

(which have slower networks), since the base styles are simpler.

Follow-up: "What's the difference between responsive and adaptive design?"

Responsive design uses fluid layouts that adjust continuously at any viewport size. Adaptive design serves distinct fixed layouts for specific breakpoints (e.g., a 320px layout, a 768px layout, a 1024px layout). In 2026, purely adaptive approaches are rare — the combination of container queries and fluid values makes responsive design the default, with adaptive-style breakpoints only for major layout shifts (e.g., switching from hamburger menu to horizontal nav).

Q 3.9 What is critical CSS and how do you extract and inline it?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart

CSS is render-blocking. The browser won't paint anything until it has downloaded and parsed all CSS linked in the `<head>`. If your CSS file is 200KB, the user sees a blank page until those 200KB download, even if only 10KB of that CSS is needed for the above-the-fold content.

Critical CSS is the minimum CSS needed to render what the user sees first (above the fold). You extract it, inline it directly in the HTML `<head>`, and defer-load the rest.

Before optimization:

HTML Copy

```
<head>
  <!-- Render-blocking - browser waits for entire 200KB file -->
  <link rel="stylesheet" href="/styles.css" />
</head>
```

After optimization:

HTML Copy

```
  <!-- Critical CSS inlined — browser can paint immediately -->
  <style>
    /* Only the CSS needed for above-the-fold content */
    body { margin: 0; font-family: Inter, sans-serif; }
    .header { display: flex; height: 64px; background: white; }
    .hero { padding: 48px 24px; background: #f8fafc; }
    .hero h1 { font-size: clamp(2rem, 5vw, 3.5rem); }
    /* ... ~10KB of critical styles ... */
  </style>

  <!-- Full CSS loaded asynchronously — non-blocking -->
  <link rel="preload" href="/styles.css" as="style"
        onload="this.onload=null;this.rel='stylesheet'" />
  <noscript><link rel="stylesheet" href="/styles.css" /></noscript>
</head>
```

The `preload` trick loads the full CSS in the background without blocking render. Once loaded, the `onload` handler switches it to a regular stylesheet. The `<noscript>` fallback ensures it works without JavaScript.

How to extract critical CSS:

Automated tools analyze your page and extract the CSS rules that apply to above-the-fold content:

- **Critters** (by Google) — A webpack/Vite plugin that inlines critical CSS at build time. Recommended for most projects.
- **Critical** (by Addy Osmani) — Generates critical CSS by rendering the page in a headless browser and extracting styles for visible elements.
- **PurgeCSS** — Different goal (removes unused CSS entirely) but often used alongside critical CSS extraction.

JAVASCRIPT Copy

```
// vite.config.js with Critters
import critters from 'critters-webpack-plugin';

export default {
```

When critical CSS matters most:

- Server-side rendered pages where First Contentful Paint (FCP) is a key metric
- Content-heavy pages (news sites, e-commerce listings) where the above-the-fold content is mostly text and images
- Pages with large CSS bundles (>50KB) where most CSS isn't needed for the initial view

When it's not worth the complexity:

- Single-page applications where the initial HTML is just a `<div id="root">` (critical CSS is meaningless if there's no server-rendered content)
- Small CSS bundles (<20KB) where the entire stylesheet loads quickly anyway
- Applications behind authentication where SEO and FCP aren't priorities

Follow-up: "How does code splitting CSS work in modern frameworks?"

Next.js and Vite automatically code-split CSS per route. Each route only loads the CSS it needs. This solves a different problem than critical CSS — it reduces total CSS per route, while critical CSS optimizes what's loaded first within a single route's CSS. In practice, route-level CSS splitting often eliminates the need for manual critical CSS extraction.

QUICK CHECK — no marks, just for you.

1. Two teams both write a `.card` class in a large codebase. According to the lesson, what actually stops those two rules from colliding under BEM, versus under CSS Modules?

- ○ BEM relies purely on naming discipline with no enforcement, while CSS Modules makes collisions impossible by hashing class names at build time
- ○ BEM scopes styles to the block element in the DOM, while CSS Modules relies on developers following the module naming convention

Neither prevents collisions; both depend on the cascade and specificity ordering to resolve duplicate class names

- Both enforce scoping automatically; BEM does it via the double-underscore syntax and CSS Modules via the .module.css file extension

Check answer

2. You have one card component rendered both in a narrow sidebar and in the wide main content area. The lesson's modern responsive approach says it should go vertical in the sidebar and horizontal in the main area. Why do container queries solve this where media queries do not?

- Container queries evaluate the viewport too, but they do it per component instead of once per page load
- Container queries respond to the container's width rather than the viewport, so the same component adapts in both places without external coordination
- Container queries have higher specificity than media queries, so the sidebar's rules override the main area's rules
- Container queries run at build time, so each placement of the card gets its own precompiled layout variant

Check answer

3. A design system implements dark mode by defining tokens on `:root` and overriding them under `[data-theme="dark"]`. A teammate asks how much of the component CSS has to change to support the new theme. Based on the lesson, what is the correct answer and why?

- Each component needs a dark variant class, because var() resolves once at parse time and will not pick up later overrides
- None — components already reference tokens via var(), and because custom properties cascade, overriding them at the root updates the whole UI

Each component needs a matching [data-theme="dark"] block, since custom properties do not cascade into child elements

- Only components using color need changes; spacing, radius and shadow tokens cannot be overridden per theme

Check answer

‹ Previous
Layout and visual

Next ›
Performance and animation

LEARN MORE

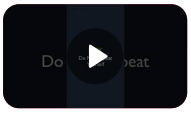
ARTICLES & DOCS

- BEM methodology
  getbem.com

VIDEOS



- BEM — the what, how & why
  YouTube



- Why you should be using BEM
  YouTube



- BEM explained
  YouTube

Hand-picked references for this lesson.